

# Taller de Programación 2

Examen Final (20 de febrero, 2024)

## Enunciado

Se desea realizar un sistema de alquiler de libros para una biblioteca con mucho público, en el que se necesita disponer de funciones que permitan implementar los siguientes requerimientos:

- Poder ingresar libros existentes para su alquiler.
- Si un libro se saca de alquiler, se eliminará de la lista de disponibilidad.
- En caso de que un ejemplar sufra algún daño se marcará como no apto, permaneciendo en el listado pero no estando disponible para su alquiler.
- La aplicación permitirá alquilar sólo los libros aptos y no alquilados.
- El sistema dispondrá de funciones que informen en forma conjunta y separada los libros disponibles para alquiler, los no aptos y los alquilados.

1. Implementar los endpoints que permitan:

- a. Dar de alta un libro, siendo sus datos: código, título, autor, estado ("disponible", "alquilado", "no-pto"). El estado al momento de ingreso será "disponible". Retornar el libro dado de alta.
- b. Dar de baja un libro por su código. Retornar el libro dado de baja.
- c. Utilizando el código del libro:
  - Alquilar el ejemplar.
  - Devolverlo.
  - Marcarlo como no apto,.

En los tres casos llevar el estado al valor correspondiente y retornar el libro modificado.

- d. Listar todos los libros y su estado.
- e. (opcional) Listar en forma separada libros disponibles, alquilados y no aptos.

2. (opcional) Al momento de alquilar, el sistema verificará con un servicio externo si el lector accede a un beneficio por sorteo que consta en quedarse con el libro alquilado. La URL externa es: <https://libros.deno.dev/premios>

Formato de la respuesta:

```
{
  sorteo: 1,           // número de sorteo ascendente
  premio: true        // true: premio otorgado, false: seguir participando
}
```

En caso de sacar el premio, el sistema dará de baja automáticamente el libro, enviando un mensaje de sorteo ganado como respuesta.

En caso de ser necesario, el servidor recibirá desde el cliente los datos requeridos en formato JSON. En caso de inconvenientes, el servidor responderá con un objeto con un campo **'errorMsg'** informando el motivo de la falla. Todas las respuestas deberán estar correctamente adosadas con su código de estado correspondiente, según el resultado de la operación. En el caso de realizar notificaciones, modularizar adecuadamente. La notificación en sí puede simularse con un mensaje por terminal.

## Aclaraciones sobre el desarrollo esperado:

1. El proyecto debe incluir únicamente el backend del sistema, utilizando Node.js + express. El formato del servidor es de tipo RESTful. Tener en cuenta los lineamientos de dicho formato, especialmente a la hora de elegir los nombres de las rutas de acceso al sistema, los verbos que accionan sobre ellas, y cómo pasar datos adicionales a la consulta.
2. El sistema debe estar correctamente separado en capas y componentes, y esta separación debe estar claramente puesta de manifiesto en la estructura de carpetas y archivos. Entre los componentes que esperamos que estén presentes encontramos: router/controlador, casos de uso, modelo/s, DAO/s, repositorios, servicios de terceros, factories, commands (los que correspondan de acuerdo al sistema modelado y se hayan visto en cursada).
3. La *validación de datos* es una parte importante del negocio, por lo tanto, observar cómo y dónde realizarla.
4. *No es necesario utilizar una conexión a base de datos real*; es posible persistir en el DAO/Repositorio usando memoria del servidor.
5. Recordar el rol de las factories, que nos permiten desacoplar las dependencias de nuestros componentes a la hora necesitar una instancia de los mismos. Recordar esto especialmente a la hora de decidir cómo obtener los casos de uso para invocarlos desde la capa de ruteo.
6. Considerar la inclusión de algún test funcional (usando axios, por ejemplo) para verificar el correcto funcionamiento de lo que se está desarrollando.
7. Pueden reutilizar código de proyectos realizados durante el cuatrimestre, siempre y cuando el código se utilice y realmente aporte al desarrollo de las funcionalidades pedidas. **La inclusión de código innecesario o fuera de lugar será penalizada.**